

Многозадачность

Многозадачность. Многопроцессорность. Виды
многозадачности. Процессы, потоки, сопрограммы.
Многозадачность в Linux.

Оглавление

[Введение](#)

[Многозадачность](#)

[Классификация по Флинну](#)

[SISD \(Single Instruction, Single Data\)](#)

[SIMD \(Single Instruction, Multiple Data\)](#)

[MISD \(Multiple Instruction, Single Data\)](#)

[MIMD \(Multiple Instruction, Multiple Data\)](#)

[Многопроцессорные системы с сильной связью](#)

[Многопроцессорные системы с гибкой связью](#)

[Программная реализация многозадачности](#)

[Однопроцессорная многозадачность](#)

[Истинная однозадачность](#)

[Однозадачность с поддержкой прерываний](#)

[Идея простейшего двухзадачного выполнения на 8086](#)

[Невытесняющая многозадачность](#)

[Вытесняющая многозадачность](#)

[Кооперативная многозадачность](#)

[Процессы, потоки и сопрограммы](#)

[Процесс](#)

[Потоки](#)

[Потоки ядра в Linux](#)

[Сопрограммы \(корутины\)](#)

[Работа с процессами и потоками в Linux](#)

[Запуск процессов в ОС Linux](#)

[Системные вызовы для управления процессами](#)

[fork\(\)](#)

[exec\(\)](#)

[wait\(\)](#)

[Жизненный цикл нового процесса](#)

[Пример жизненного цикла](#)

[Процесс-зомби и процесс-сирота](#)

[Мониторинг ресурсов с помощью top](#)

[Отправка сигналов процессам](#)

[Процессы и потоки в программировании](#)

[Работа с процессами на C](#)

[Потоки в C](#)

[Практическое задание](#)

[Дополнительные материалы](#)

[Используемая литература](#)

Введение

Прежде чем перейти к многозадачности, необходимо понять, что такое программа и процесс. Процесс — это программа, которая выполняется в данный момент. Если программа — это алгоритм, реализованный в машинном коде или на языке высокого уровня, но все равно либо скомпилированный в машинный код, либо выполняемый интерпретатором, то процесс — запущенная, выполняемая программа. При этом, если программа написана в машинном коде (либо на C, C++, Go и откомпилирована в машинный код), то ее инструкции, выполняемые процессором, и образуют процесс. Если же программа написана на языке интерпретируемом или скриптовом (Python, Javascript, bash), то процесс — это выполняющийся код интерпретатора, который уже исполняет код языка программирования. Процесс также включает не только код, но и ресурсы, такие как сам код (программу), память, которая содержит как код, так и данные, переменные окружения, стек вызовов, heap-кучу — область памяти для хранения создаваемых в процессе работы переменных, дескрипторы открытых файлов и сокетов, права и идентификаторы владельца процесса, группы процесса, контекст — состояние процессора (регистры, схему преобразования виртуальных адресов в физические, уровень выполнения). При запуске процесса он получает аргументы командной строки и переменные окружения операционной системы. При завершении процесса он возвращает родительскому процессу код возврата. 0 — означает, что все завершилось успешно. Если число, то оно трактуется как код ошибки.

Когда код лежит на диске в виде исполняемого файла — это программа. Когда код запущен на выполнение и использует ресурсы ЭВМ — это процесс.

Многозадачность

Самый простой способ исполнять задачи — последовательный. Мы запускаем программу, процесс отработывает, завершает исполнение, вызывается новый. Но даже в последовательном выполнении можно видеть предшественников многозадачности. Программа запускается, работает, далее из запущенной программы запускается дочерняя, и пока действует дочерний процесс, родительский ничего не делает. После того как дочерний процесс завершил работу, управление снова передается родительскому. Но иногда возникает потребность выполнять задачи параллельно.

Самый простой способ выполнить две задачи — это запустить две программы на двух разных ЭВМ. Но чтобы это была настоящая многозадачность, нужно обеспечить управление запуском программ, чтобы эти две ЭВМ работали как одна. Кстати, некоторые суперкомпьютеры так и работают (состоят из множества материнских плат с собственными процессорами и оперативной памятью и синхронизируются благодаря высокоскоростной Ethernet-сети).

А как же быть владельцам самых простых систем, с одним процессором, более того — с одноплатным? (Многоядерные процессоры на самом деле содержат внутри несколько независимых процессоров, объединенных в одном корпусе и тесно связанных между собой. Стоит различать многопроцессорные компьютеры, где процессоры отдельные, и многоядерные, где процессор один, но включает несколько ядер. И многопроцессорный, и многоядерный компьютеры могут физически выполнять код параллельно.)

Даже на одноплатном процессоре возможно псевдопараллельное выполнение разных задач — путем выделения каждому процессу кванта времени, с переключением между этими квантами. Для пользователя это выглядит как выполнение двух и более задач одновременно.

Не всегда нужно параллельное выполнение двух задач. Например, если запустить текстовый редактор и командную строку, то пользователь будет переключаться между ними, но не работать одновременно. Логично, что в таком случае не нужно выделять кванты времени по очереди каждому процессу. Можно давать на выполнение код программы в соответствии с тем, как между задачами переключается сам пользователь. Например, пользователь работает в текстовом редакторе. Нажимает кнопку переключения задач. Текстовый редактор приостанавливается, но остается в оперативной памяти. С

точки зрения ОС этот процесс «засыпает». Но при переключении в терминал его работа возобновляется. При следующем переключении — наоборот.



Суперкомпьютер Summit — один из самых мощных современных суперкомпьютеров. [Автом: Carlos Jones/ORNL](#)

Есть разница между последовательным запуском задач (так было в DOS) и переключаемой (вытесняющей) многозадачностью. Если в DOS из командной строки зашли в текстовый редактор, то снова зайти в командную строку можно было либо полностью сохранив файл и выйдя из текстового редактора, либо из текстового редактора запустив вложенную командную строку как новый процесс. И только завершив вложенный процесс, можно было вернуться в редактор. В первых системах с вытесняющей многозадачностью, таких как Dos Shell, появилась возможность переключаться между двумя задачами по кнопке (как правило, Tab) без завершения каждого из процессов.

Многозадачность — это признак операционной системы. Если ОС умеет работать с многозадачностью, она это будет делать и на одноядерном процессоре. Процессор либо ядро процессора выполняет за одну единицу только одну инструкцию.

Если же нет, то будет возможно строго последовательное выполнение задач (но и такой подход тоже оправдан для соответствующих задач, например программ-прошивок, непосредственно исполняющихся в микроконтроллерах).

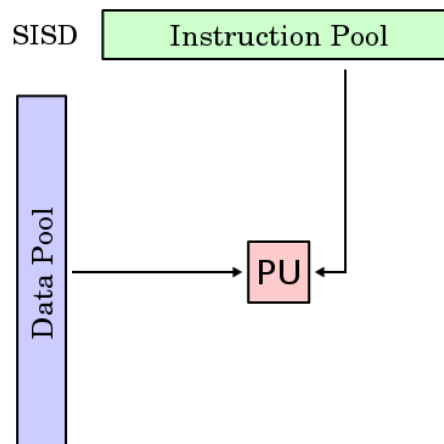
Прежде чем перейти к программным реализациям многозадачности, рассмотрим классификацию архитектур ЭВМ по признакам наличия параллелизма в потоках данных и команд, предложенную Майклом Флинном в 1966 году.

Классификация по Флинну

Поскольку каждый процессор работает с потоком команд и потоком данных, по способу распределения ресурсов выделяют несколько способов организации.

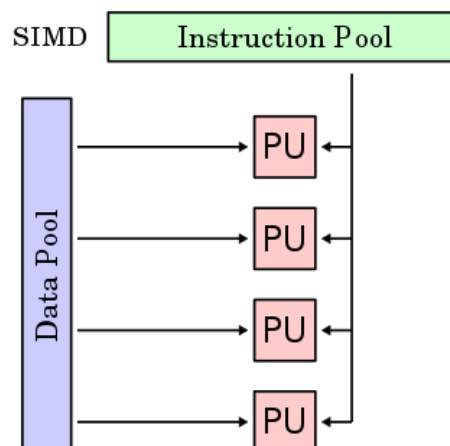
SISD (Single Instruction, Single Data)

SISD (Single Instruction, Single Data) — один поток команд, один поток данных. Эта традиционная однопроцессорная архитектура задействует практически все современные процессоры. При этом SISD-процессоры могут быть объединены в многопроцессорные системы.



SIMD (Single Instruction, Multiple Data)

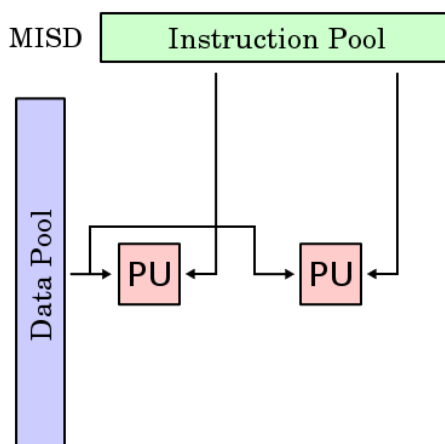
SIMD (Single Instruction, Multiple Data) — один поток команд, множество потоков данных. Этот способ позволяет осуществлять параллельную обработку данных, но одной программой и реализуется и в векторных, и в графических процессорах. При этом для работы с вещественными числами SIMD-расширения имеются в современных процессорах (MMX, SSE и т. д.).



MISD (Multiple Instruction, Single Data)

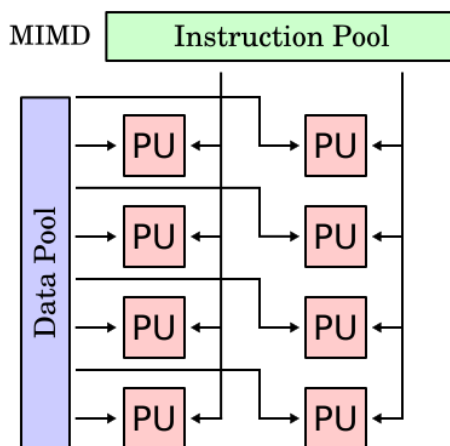
MISD (Multiple Instruction, Single Data) — множество потоков команд, один поток данных. Фактическая польза от такой архитектуры — резервирование. Она может применяться в военной, аэрокосмической

сфере, управлении технологическими процессами и т. д., где важна отказоустойчивость и продолжение работы даже при отказах оборудования.



MIMD (Multiple Instruction, Multiple Data)

MIMD (Multiple Instruction, Multiple Data) — множество потоков команд, множество потоков данных. Каждый процессор независимо выполняет команды и обрабатывает данные, при этом одновременно разными процессорами могут выполняться разные фрагменты одной и той же программы и обрабатываться разные фрагменты одних и тех же данных.



Также выделяют классификацию многопроцессорных систем по степени связи.

Многопроцессорные системы с сильной связью

Многопроцессорные системы с сильной связью (Tightly-coupled multiprocessor systems) содержат несколько процессоров, соединенных на шинном уровне. Эти процессоры могут иметь доступ к центральной разделяемой памяти (SMP или UMA) или участвовать в иерархии памяти и с локальной, и с разделяемой памятью (NUMA). Пример процессора, ориентированного на многопроцессорные системы, — Intel Xeon: у каждого процессора свой кеш, но доступ к разделяемой памяти через общую шину.

Предельная форма многопроцессорных систем с сильной связью — многоядерные процессоры, где несколько процессов реализуются в одном чипе.

Многопроцессорные системы с гибкой связью

Многопроцессорные системы с гибкой связью (Loosely-coupled multiprocessor systems), часто называемые кластерами, основаны на множественных автономных одиночных или двойных компьютерах, связанных через высокоскоростную систему связи (например, Gigabit Ethernet).

Как правило, в таких системах многопроцессорность реализуется на более высоком уровне. Межпроцессное взаимодействие выполняется с применением стека протоколов TCP/IP, содержащего компоненты межпроцессного взаимодействия. Вместе с технологиями виртуализации это дает гибкий, масштабируемый и более дешевый подход по сравнению с многопроцессорными системами с сильной связью. Как правило, компоненты многопроцессорных систем с гибкой связью вне системы могут функционировать как отдельные компьютеры.

Программная реализация многозадачности

Многозадачность следует отличать от многопроцессорности: хотя многозадачность может быть реализована (и реализуется) с применением нескольких процессоров, но в общем случае это возможно и на одном процессоре. Более того, в неявном виде она существует и в однозадачных операционных системах, поскольку в них реализован механизм прерываний.

Механизм прерываний, который поддерживается процессорами аппаратно и позволяет переключаться с выполнения процесса на выполнение кода-обработчика программы с сохранением контекста программы, которая выполнялась ранее, — это и есть истинная основа программной многозадачности. И сейчас технически многозадачность реализуется с помощью прерываний (по крайней мере, когда речь идет о процессах и некоторых реализациях потоков, как в Linux). Таким образом, если процессор компьютера поддерживает прерывания, то для такого компьютера может быть написана многозадачная операционная система.

История знает и исключение — когда-то для радиолюбителей был доступен компьютер Радио 86РК, который можно было собрать по чертежам с использованием дампов прошивок, публикуемых в советских журналах: это в полном смысле однозадачный компьютер. Во время операций ввода-вывода, например при загрузке программы с ленты, экран монитора гас. Интересно, что Радио 86РК базировался на отечественных клонах Intel 8086, но прерывания в машине не использовались. Сам вывод INT процессора использовался для управления звуком.

Если в системе есть механизм прерываний, то даже однозадачная операционная система на самом деле работает с несколькими задачами, между которыми происходит переключение. Это код выполняемой в данный момент единственной задачи и код прерываний, который по определенному событию прерывает выполнение программы, — процессор выполняет код обработчика прерывания и возвращает управление прерванной программе. Впрочем, механизм прерываний позволяет реализовать и более привычную для пользователя многозадачность. Хотя качество такой реализации зависит также от возможностей процессора — в частности, от возможности защиты кода и данных. В процессоре 8086 таких механизмов не было — они впервые появились в 80286, а полноценно были реализованы в Intel 80386.

Однопроцессорная многозадачность

Истинная однозадачность



Абсолютно однозадачными были калькуляторы на базе процессора Intel 4004. Процессор не имел поддержки прерываний — она появилась в Intel 4040.



Радио 86 РК (изображение с сайта <http://sfrolov.livejournal.com/79155.html>)

Однозадачным был «Радио 86 РК», несмотря на поддержку прерываний в процессоре 8086. Вывод процессора INT использовался для формирования звука. Во время загрузки программы с магнитофонной ленты экран гас.

Похожее наблюдалось и у компьютера ZX-80: построенный на базе процессора Z80 с поддержкой прерываний, он был устроен таким образом, что даже генерация изображения осуществлялась программно. Таким образом, и при нажатии кнопки, и при выполнении BASIC-программы экран гас.

Однозадачность с поддержкой прерываний

Поддержка прерываний позволяет эффективно реализовать работу с внешними устройствами. Процессор не должен читать порт клавиатуры и заниматься отрисовкой видео. Аппаратные прерывания позволяют выполнять небольшие фрагменты кода обработчика прерывания, накапливая, например, полученные коды символов от клавиатуры в буфер, а при чтении соответствующей функции ПЗУ — передавать их программе.

Компьютер становится значительно более интерактивным, чем его бытовые предшественники. Тем не менее задачи могут выполняться только по очереди. Если в MS DOS изначально запущен COMMAND.COM, при запуске программы (например, игры), чтобы вернуться в командный интерпретатор, приходилось завершить программу. Это неудобно, поэтому программы либо

имитировали командную строку сами (как это делали многочисленные командеры, такие как Norton Commander, Volkov Commander, Dos Navigator), либо реализовали «выход в DOS» (QBASIC, Turbo Pascal, Lexicon), попросту запуская новый экземпляр command.com и оставаясь в памяти, но ничего не делая. Полноценной многозадачности здесь нет, даже невытесняющей: одна программа просто запускает другую, однако при этом уже выполняется не один код. Если нажать правый Ctrl, у экрана появляется красная рамка и регистр переключается с латиницы на русский алфавит — это сработал обработчик прерывания по клавиатуре драйвера-русификатора keyrus. Если переключить снова, программа опять на время прерывается: выполнен код. Таким образом фактически параллельно работают две программы: запущенная программа (редактор NCEDIT.EXE из состава Norton Commander или встроенный в DOS EDIT.COM, а на самом деле QBASIC.EXE /EDCOM) и переключатель keyrus.

Идея простейшего двухзадачного выполнения на 8086

Как быть, если в программе нет выхода в DOS? Можно написать обработчик прерывания, перехватить обработчик прерывания клавиатуры и, отследив нажатие одной фиксированной комбинации кнопок (например, Alt-Shift), запустить COMMAND.COM. Разумеется, необходимо сохранить состояние экрана, чтобы восстановить его после выхода из COMMAND.COM, а также сохранить и восстановить значения регистров процессора. Вместо COMMAND.COM можно запустить другую программу — например, VC.COM. Если же нажата не заданная комбинация Alt-Shift, управление передаем исходному обработчику прерываний клавиатуры.

В таком случае о передаче управления по-прежнему речи не идет, но ее можно реализовать — тоже через прерывания. Например, запустим исходное приложение (редактор, игру), нажмем Alt-Shift и проверим состояние системных переменных. Допустим, одна из них хранит статус, показывающий, какая программа сейчас выполняется (исходная — 0 или «вторая» — 1), а вторая — запущена ли вторая программа (0 — не запущена, 1 — запущена). Соответственно, если выполняется первая программа и вторая не запущена, запускаем вторую программу и устанавливаем системные переменные в значения 1 и 1. Запускается command.com, и мы как будто «выходим в DOS». Но если опять нажать Alt-Shift, снова запускается обработчик прерывания. Так как выполняется вторая программа (1 — «вторая», 1 — запущена), меняем статус (0 — «исходная», 1 — запущена) и возвращаем управление в первую программу. Для этого необходимо восстановить состояние исходной программы — в частности, восстановить экран и поместить в стек исходный адрес возврата, чтобы по выходе из прерывания произошел возврат не во вторую программу, а в первую. Независимо от того, из какой и в какую программу мы переходим, приходится сохранять состояния стека, экрана и адрес возврата — указание на место, с которого мы продолжим выполнение программы.

После выхода из command.com необходимо установить соответствующую переменную (0 — исходная, 0 — не запущена) и вернуть управление в исходную программу. Можно реализовать и более сложную схему с поочередным переключением между несколькими программами. Впрочем, на процессоре 8086 реализовать полноценную многозадачность не получится по двум причинам.

Первая — недостаток памяти, который не позволяет запускать две сложные программы: при таком запуске получается, что функционирует одна программа и нечто вспомогательное, вроде «выхода в DOS», но только по кнопке.

Вторая — отсутствие механизмов защиты. Неважно, предоставлен ли механизм «выход в DOS» самой программой или мы его реализовали через прерывание и даже возможность переключения между двумя задачами. Если запустить в режиме COMMAND.COM программу с ошибкой (например, переименовать расширение текстового файла в .com или .exe и запустить), система останавливается, и данные в первой программе могут быть потеряны. При переключении между задачами иногда еще можно было переключиться в исходную программу (без возможности остановить вторую), а иногда нет — например, если сбойная программа отключила прерывания.

Невытесняющая многозадачность

Дальнейшее развитие замысла — идея невытесняющей многозадачности. При соответствующей поддержке ОС (DR DOS 6, проект MS DOS 4) или программы (DESQView, DOSShell) появляется возможность запускать несколько программ и выполнять их по очереди. В один момент выполняется только одна задача, в момент запуска остальные приостанавливаются. Переключение пользователь выполнял, как правило, нажатием сочетания клавиш Alt-Tab.

Те, кто знаком с играми, могут сравнить невытесняющую многозадачность с пошаговой стратегией (TBS — Turn Based Strategy). Так же, как в игре юниты двигаются строго последовательно, процессы при невытесняющей многозадачности работают строго последовательно. Плеер при невытесняющей многозадачности не сможет играть музыку, пока не активен!



В правом нижнем углу список процессов для переключения в системе Dos Shell — образец применения невытесняющей многозадачности.

Для полноценной реализации механизма требовался достаточный объем памяти, поэтому полноценно такие режимы были реализованы на процессоре 80386 (DESQView с помощью QEMM386, EMM386 для MS DOS и DR DOS).

Разумеется, для полноценной мультизадачности требовалась возможность параллельного выполнения задач.

Вытесняющая многозадачность

Казалось бы, на одном процессоре невозможно решить проблему многозадачности комфортно для повседневной работы. Но что если осуществлять переключение между задачами не по редко

возникающему событию (нажатию кнопки), а по таймеру или при поступлении любого прерывания? И делать это настолько часто, чтобы для пользователя выглядело, будто обе программы работают одновременно. Тогда каждая будет получать квант времени и возникнет иллюзия параллельности. В современных операционных системах реализован именно такой подход.

Те, кто знаком с играми, могут сравнить вытесняющую многозадачность со стратегией реального времени (RTS — Real Time Strategy). Как в игре юниты двигаются независимо и одновременно, так и процессы при вытесняющей многозадачности работают параллельно. На самом деле компьютер просчитывает юниты по очереди, и процессы выполняют кванты кода по очереди, но для пользователя это не заметно — создается иллюзия одновременности. Плеер при вытесняющей многозадачности сможет играть музыку, даже пока неактивен, а вы печатаете в Microsoft Word!

Обратите внимание: процессор при этом по-прежнему ничего не знает о многозадачности — он выполняет текущую операцию, а управление переключением осуществляет операционная система.

Современные операционные системы поддерживают вытесняющую многозадачность и реализуют ее. Параллельное прослушивание музыки в одной программе, загрузка файлов в другой и чтение веб-страницы — это она, вытесняющая многозадачность.

Кооперативная многозадачность

В этом режиме переключением задач занимается не планировщик, а частично сами задачи. Чтобы задача переключилась, предыдущая должна освободить ресурсы и отдать управление. Кооперативная многозадачность применяется между потоками ядра Linux и применялась в Windows 3.x. В качестве ядра данной системы использовалась MS-DOS,

При использовании процессора 80386 в Windows 3.x использовалась комбинация невытесняющей и кооперативной многозадачности. Процессами в рамках невытесняющей многозадачности являлась сама Windows 3.X и DOS-приложения в виртуальных 8086-машинах, поддерживаемых процессором. Сами же Windows-приложения работали по принципу кооперативной многозадачности.

Современные языки программирования позволяют реализовывать кооперативную многозадачность своими силами. В этом помогают корутины в Kotlin и горутины в Go — реализации сопрограмм (coroutines).

Процессы, потоки и сопрограммы

Есть четыре вида организации параллельно выполняемого кода:

- процессы;
- потоки;
- процессы (потоки) ядра;
- корутины (сoproграммы).

Процесс

Обладает собственными ресурсами. При использовании механизмов защиты памяти процессором процесс изолирован от других процессов. Он работает в виртуальной памяти, не имея доступа к ресурсам других процессов. Если два процесса подразумевают общение, они должны это делать через API операционной системы, как правило через файлы или сокеты.

Самый простой способ общаться — когда одно приложение пишет в файл, а второе читает. Но современные операционные системы предоставляют более удобные механизмы, чем использование жесткого диска, — это буферы, которые выглядят для программы как файл.

В ОС Linux есть специальный вид файла FIFO, который позволяет создать буфер. Но при работе с ним как с файлом два приложения не будут писать на диск, а одно сразу будет отправлять сообщение другому.

Создадим FIFO и посмотрим атрибуты (см. терминал № 0 на картинке ниже):

```
$ mkfifo buffer
$ ls -la buffer
```

Далее в одном окне терминала мы пишем (назовем его терминалом № 1, см. картинку ниже):

```
$ cat > buffer
```

А во втором (терминал № 2):

```
$ cat < buffer
```

После этого все, что мы пишем в терминале № 1, отображается в терминале № 2.

Потому что мы запустили два разных процесса **cat**, между которыми возможно общение.

Если мы запустим эту команду:

```
ps ax|grep cat
```

... то увидим, что это два разных, независимых процесса.

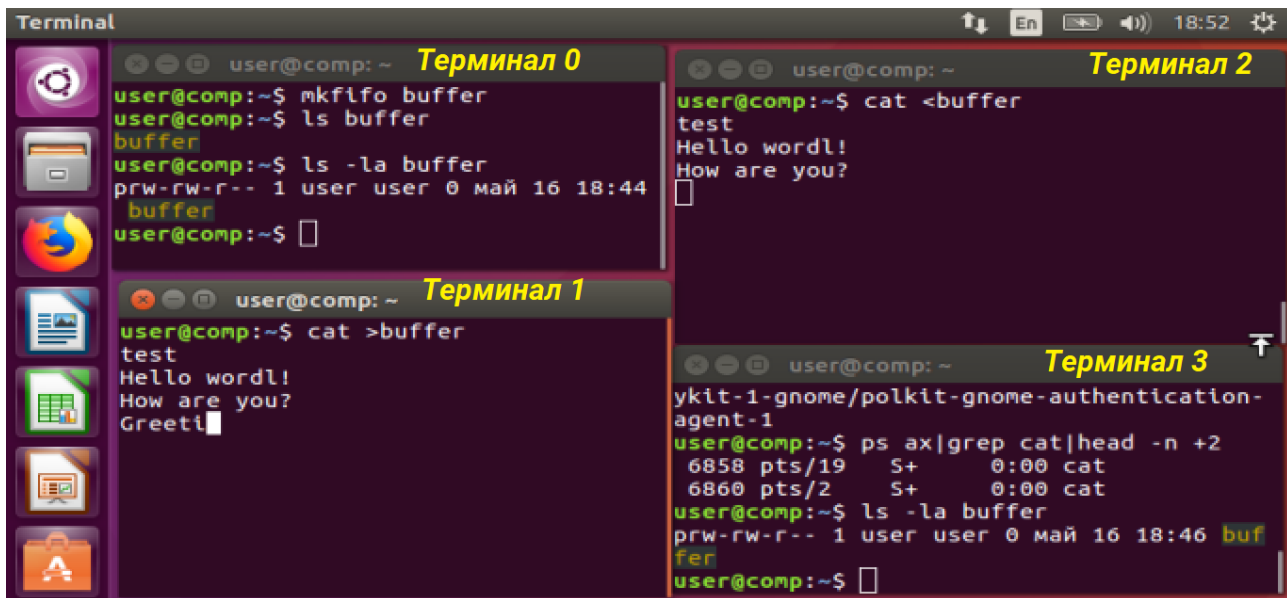
Команда **ps ax** вывела список процессов, затем отдала результаты команде **grep** (это еще один способ межпроцессного взаимодействия), которая вывела только строки, содержащие слово **cat**. В терминале № 3 на рисунке также отображен третий участник конвейера — команда **head**, которая служит для того, чтобы из всего вывода отфильтровать только первые две строки.

Если мы снова прочитаем атрибуты файла **buffer**:

```
$ ls -la buffer
```

(см. терминал № 3 на картинке),

... то увидим, что буфер не занимает место на диске.



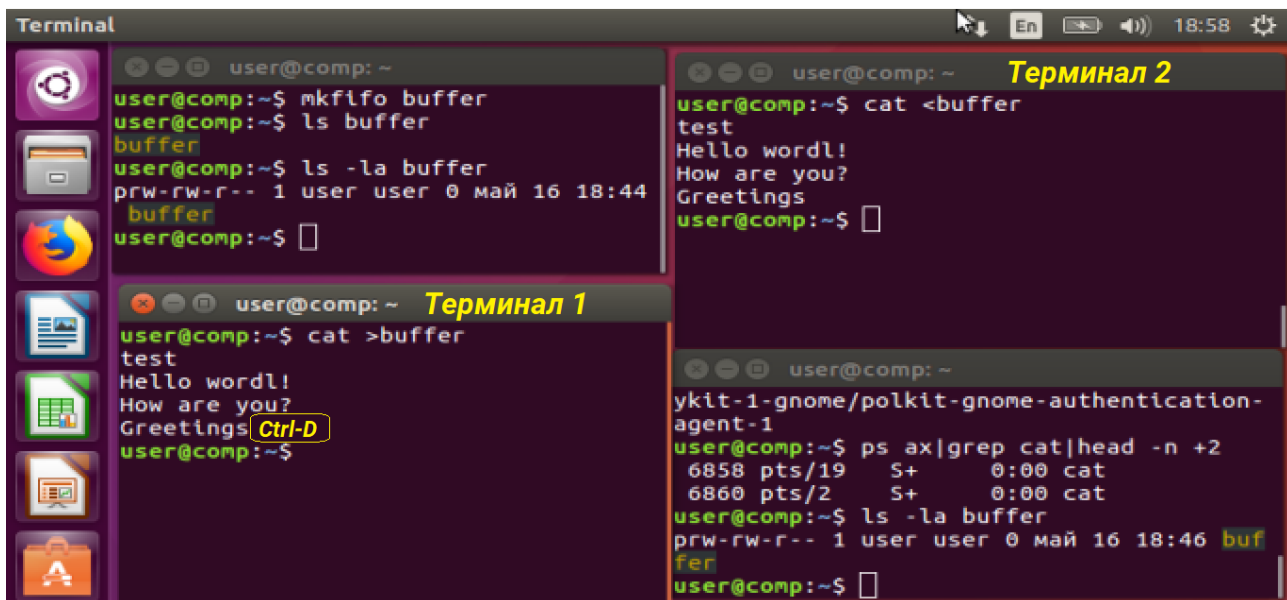
Чтобы завершить связь, достаточно в терминале № 1 нажать сочетание клавиш Ctrl-D (это символ конца файла. Обратите внимание, что в Windows похожую роль играет комбинация Ctrl-Z, а вот в Linux Ctrl-Z — означает «приостановить процесс»). Так как мы в Linux, жмем Ctrl-D (см. следующий рисунок).

При этом в данном случае первый процесс **cat** завершается, так как больше нужды писать в файл нет — его задача выполнена.

Второй процесс **cat** тоже завершается, так как при чтении достигнут конец файла.

Но этим же методом могут пользоваться и любые другие две программы.

Альтернатива — использование TCP-сокетов с использованием **localhost**, то есть адреса 127.0.0.1 (или любого другого в сети 127.0.0.0/8), либо использование UNIX-сокетов.



Так же, как с файлами, процессы общаются с устройством ввода-вывода — терминалом (когда-то он был отдельным сетевым устройством).



Телетайп из 80-х, использовался для работы на PDP 11-05

Сейчас терминал в качестве устройства представлен комбинацией «монитор + клавиатура».

В DOS и Windows с терминалом можно общаться, используя имя файла **con**.

Копируя файл **con** в файл **test.txt** командой:

```
c:\> copy con test.txt
```

... мы будем считывать строки с клавиатуры и записывать в файл. Завершить ввод можно, введя комбинацию символа конца файла (EOF) — в Windows это Ctrl-Z.

Команда копирования файла **test.txt** в файл **con**:

```
c:\> copy test.txt con
```

Выводит на экран текст файла **test.txt**.

В Linux и Mac OS этого можно добиться командой **cp** и использованием устройства **/dev/tty** (или **/dev/pts** — в зависимости от системы).

Копируя файл **/dev/tty** в файл **test.txt** командой:

```
$ cp /dev/tty test.txt
```

... мы будем считывать строки с клавиатуры и записывать в файл. Завершить ввод можно, введя комбинацию символа конца файла (EOF) — в Linux это Ctrl-D (и ни в коем случае не Ctrl-Z, который останавливает выполнение процесса, но не завершает его).

Команда копирования файла **test.txt** в файл **/dev/tty**:

```
$ cp test.txt /dev/tty
```

Выводит на экран текст файла **test.txt**.

Есть в UNIX-подобных системах и отдельные специальные файлы.

/dev/stdin — стандартный входной поток, как правило, клавиатура.

/dev/stdout — стандартный выходной поток, как правило, экран.

/dev/stderr — стандартный поток ошибок, по умолчанию выводится на экран.

Когда программа хочет вывести сообщение на экран, она его записывает в файл **/dev/stdout**, а чтобы прочитать данные с клавиатуры — читает **/dev/stdin**.

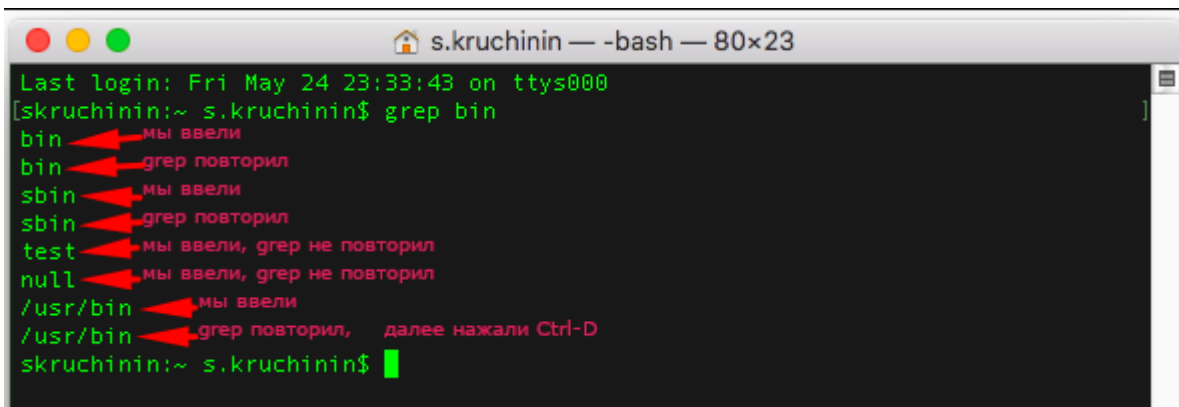
Благодаря тому, что программы используют стандартные вход и выход, можно перенаправлять потоки ввода-вывода.

Рассмотрим пример в Linux или Mac OS.

Выполним в терминале команду:

```
grep bin
```

Запущенный командой **grep** процесс получит в качестве аргумента командной строки первый (и в данном примере единственный) параметр **bin**. Процесс будет на **stdout** отдавать только те строки, которые содержат подстроку **bin**, указанную в параметре. Но исходный набор строк будет ожидать с **stdin**, то есть с клавиатуры. Для завершения ввода нужно будет нажать Ctrl-D.



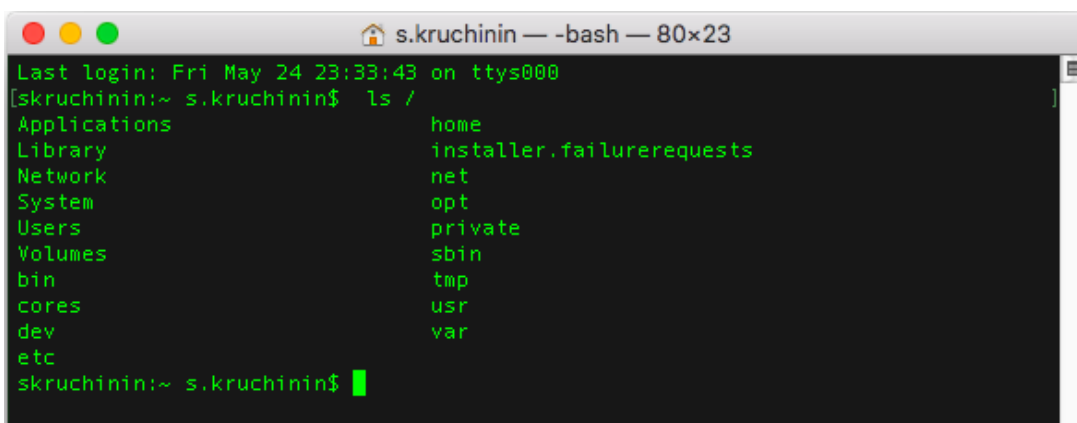
```
s.kruchinin — -bash — 80x23
Last login: Fri May 24 23:33:43 on ttys000
[skruchinin:~ s.kruchinin$ grep bin
bin
sbin
test
null
/usr/bin
/usr/bin
skruchinin:~ s.kruchinin$
```

Annotations in the image: Red arrows point to the input lines. Text next to arrows explains: 'мы ввели' (we entered) for the first input, 'grep повторил' (grep repeated) for subsequent inputs, and 'grep не повторил' (grep did not repeat) for inputs that did not match. The final input is followed by 'далее нажали Ctrl-D' (then we pressed Ctrl-D).

Если же мы выполним команду:

```
ls /
```

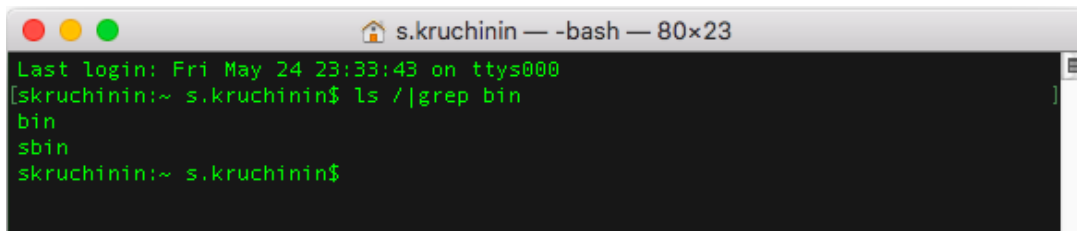
... то запущенный ею процесс не будет читать **stdin**. Ему будет достаточно первого (и в данном примере единственного) аргумента командной строки — **/**, то есть имени корневой директории, содержимое которой процесс **ls** запишет в файл **stdout**, фактически выводя на экран.



```
s.kruchinin — -bash — 80x23
Last login: Fri May 24 23:33:43 on ttys000
[skruchinin:~ s.kruchinin$ ls /
Applications      home
Library           installer.failurerequests
Network           net
System            opt
Users             private
Volumes           sbin
bin               tmp
cores            usr
dev              var
etc
skruchinin:~ s.kruchinin$
```

Но мы можем соединить два процесса таким образом, что выдаваемый в **stdout** текст одного процесса (**ls /**) будет передаваться на **stdin** другого процесса (**grep bin**). Делается это с помощью символа **|**:

```
ls / | grep bin
```

A terminal window titled 's.kruchinin — -bash — 80x23'. The prompt is 'skruchinin:~ s.kruchinin\$'. The command 'ls / | grep bin' has been entered and executed. The output is 'bin' and 'sbin'. The prompt is now 'skruchinin:~ s.kruchinin\$'.

Процессу **grep** не пришлось читать данные с клавиатуры. Он прочитал данные, которое направлял в **stdout** процесс **ls**.

Такой механизм называется **конвейером**.

Также можно заменить **stdin** и **stdout** на другой файл.

Записать вывод команды **ls /** в файл **rootdir.txt** (если файл существовал, будет перезаписан):

```
ls / > rootdir.txt
```

Записать вывод команды **ls ~** в конец файла **rootdir.txt** (если файл существовал, будет дописано в конец):

```
ls ~ >> rootdir.txt
```

Вместо **stdin** заставить **grep** читать файл **rootdir.txt**:

```
grep bin < rootdir.txt
```

Такой механизм называется перенаправлением потоков ввода-вывода.

Аналогично работает и в UNIX-системах, и в DOS и в Windows.

Потоки

Если процесс самостоятельно распоряжается памятью при создании форка и память копируется, то поток похож на процесс. Но все потоки работают с одной памятью, представленной процессом.

Потоки также иногда называют нитями (**threads**).

Реализация потоков, как и процессов, определяется операционной системой. Если в Windows процесс — контейнер потоков, то в Linux поток, по существу, реализован так же, как процесс.

В соответствии со стандартом POSIX у всех потоков должен быть один идентификатор процесса. В Linux это достигнуто через ухищрение: в качестве идентификатора возвращается идентификатор первого процесса многопоточного приложения.

Потокам нет необходимости использовать такие же механизмы обмена, как с процессорами, для общения между собой (для общения с другими процессами такая потребность есть).

Потоки могут использовать общую память. Но чтобы потоки могли не испортить общие ресурсы, они должны быть написаны соответствующим образом. А когда нужно использовать общий ресурс, то потоки используют семафоры и мьютексы.

Мьютекс — признак, позволяющий разрешить или запретить потоку доступ к ресурсу. Если один поток использует ресурс, то мьютекс занят, и второй поток не использует этот ресурс до освобождения мьютекса.

Семафор, в отличие от мьютекса, позволяет дать доступ нескольким потокам. Семафор использует счетчик. Если счетчик равен нулю, доступ запрещен. Если больше нуля — разрешен.

Потоки ядра в Linux

Компоненты ядра в Linux, работая в пространстве ядра, в отличие от процессов и потоков, работающих в пользовательском пространстве, также действуют «параллельно». Они называются процессами ядра, или потоками ядра. В данном случае это не столько техническая характеристика (ведь потоки в Linux также являются особым видом процессами), но и подчеркивание того, что в пространстве ядра потоки ядра обладают общими ресурсами, и не изолированы друг друга, в отличие от пользовательских процессов.

По умолчанию **ps** отображает процессы пользовательского пространства и потоки ядра (отображаются в квадратных скобках). Потоки не отображаются — чтобы посмотреть их, используйте ключ **-T**. Увидеть потоки конкретного процесса можно, используя ключ **-p**:

```
$ ps -T -p 4242
```

Сопрограммы (корутины)

В отличие от процессов и потоков, реализуемых операционной системой, сопрограммы (корутины, горутины) реализуются самой программой (компилятором, библиотеками, архитектурой языка).

Сопрограммы реализуют пример кооперативной многозадачности. То есть спроектированы таким образом, чтобы самостоятельно передавать управление друг другу. Сопрограмма приостанавливается в определенной точке, сохраняя полное состояние (включая стек вызовов и счетчик команд), и передает управление другой сопрограмме. Та, в свою очередь, выполняет задачу и передает управление обратно, сохраняя свои стек и счетчик.

Сопрограммы есть не во всех языках. В наиболее молодых они встроены в язык, в Go — горутины, в Kotlin — корутины.

В C и C++ корутин нет, но аналогичную работу может реализовать программист — с помощью флагов или инструкции **switch**. Но, как правило, используют похожие на сопрограммы потоки (хотя эти механизмы и более низкоуровневые и затратные по сравнению с корутинами).

Работа с процессами и потоками в Linux

Современные операционные системы состоят как минимум из двух сильно отличающихся по своим возможностям частей. Это **ядро**, работающее в пространстве ядра и на уровне режима работы процессора имеющее возможность работать со всем оборудованием и ресурсами, и **пользовательское пространство**, работающее в уровне ядра, а на уровне режима работы процессора ограниченное в доступе к ресурсам. Попытки обратиться к неразрешенным ресурсам приводят к вызову процессором исключения и передаче управления ядру операционной системы.

Так как Linux — система многозадачная, существует три возможности реализации многозадачности. Одна — для ядра, и две — для пользовательского пространства.

Для ядра это потоки (процессы) ядра.

Для пользовательского пространства — процесс и потоки. В Linux потоки реализованы таким образом, что также являются особым рода процессами (они тоже имеют собственные PID). Но при этом они делят одну и ту же память процесса, которому принадлежат (первый созданный процесс, фактически он же — первый поток процесса). При создании копии процесса (**fork**) происходит копирование его ресурсов, поэтому это более безопасно, но менее эффективно.

И процесс, и потоки, и потоки ядра имеют единую нумерацию **PID (process ID)**. При этом потоки имеют аналогичный PID. Из-за особенностей реализации потоков PID процесса считается PID первого его потока.

Каждый процесс имеет ряд ID: идентификатор владельца процесса UID, идентификатор группы владельца процесса GID, идентификатор процесса PID (process ID) и идентификатор родителя PPID (parent process ID).

Перечень процессов можно посмотреть с помощью команды:

```
$ ps aux
```

Если мы запустим:

```
$ ps alx
```

... то для каждого процесса увидим PID (id процесса) и PPID (id родительского процесса).

Для более удобного просмотра можно открыть так:

```
$ ps alx|less
```

Рассмотрим пример вывода.

В квадратных скобках — процессы ядра.

	UID	PID	PPID	PRI	NI	VSZ	RSS	WCHAN	STAT	TTY	TIME	COMMAND
4	0	1	0	20	0	185184	3952	-	Ss	?	0:27	/lib/systemd/systemd --system -
1	0	2	0	20	0	0	0	-	S	?	0:00	[kthreadd]
1	0	4	2	0	-20	0	0	-	S<	?	0:00	[kworker/0:0H]
1	0	6	2	0	-20	0	0	-	S<	?	0:00	[mm_percpu_wq]
1	0	7	2	20	0	0	0	-	S	?	0:03	[ksoftirqd/0]
1	0	8	2	20	0	0	0	-	S	?	0:05	[rcu_sched]
1	0	9	2	20	0	0	0	-	S	?	0:00	[rcu_bh]
1	0	10	2	-100	-	0	0	-	S	?	0:00	[migration/0]
5	0	11	2	-100	-	0	0	-	S	?	0:01	[watchdog/0]
1	0	12	2	20	0	0	0	-	S	?	0:00	[cpuhp/0]
5	0	13	2	20	0	0	0	-	S	?	0:00	[kdevtmpfs]
1	0	14	2	0	-20	0	0	-	S<	?	0:00	[netns]

Пролистываем вниз:


```

0 1000 31823 31183 20 0 357616 724 poll_s S1 ? 0:00 /usr/lib/gvfs/gvfsd-trash --spa
wner :1.9 /org/gtk/gvfs/exec_spaw/0
0 1000 31884 31183 20 0 1246536 103348 poll_s Ss1 ? 45:42 compiz
0 1000 31905 31492 20 0 493856 2804 poll_s S1 ? 0:00 zeitgeist-datahub
0 1000 31912 31183 20 0 4504 72 wait S ? 0:00 /bin/sh -c /usr/lib/x86_64-linu
x-gnu/zeitgeist/zeitgeist-maybe-vacuum; /usr/bin/zeitgeist-daemon
0 1000 31916 31912 20 0 410192 4260 poll_s S1 ? 0:00 /usr/bin/zeitgeist-daemon
0 1000 31924 31183 20 0 317224 6216 poll_s S1 ? 0:00 /usr/lib/x86_64-linux-gnu/zeitg
eist-fts
0 1000 31941 31183 20 0 672780 22684 poll_s S1 ? 0:07 /usr/lib/gnome-terminal/gnome-t
erminal-server
0 1000 31948 31183 20 0 68236 1916 poll_s S ? 0:00 /usr/lib/x86_64-linux-gnu/gconf
/gconfd-2
0 1000 31953 31941 20 0 29496 2644 wait_w Ss+ pts/2 0:00 bash
0 1000 31984 31492 20 0 604940 14420 poll_s S1 ? 0:00 update-notifier
0 1000 32331 31492 20 0 516252 2492 poll_s S1 ? 0:00 /usr/lib/x86_64-linux-gnu/deja-
dup/deja-dup-monitor
(END)_

```

Поле **STAT** отображает состояние процесса.

Состояния могут быть следующие:

Отображени е в ps	Состояние	Описание
R	Runnable	Работающий процесс (выполняется в данный момент либо ожидает очереди для выполнения)
S	Sleeping	Процесс в состоянии ожидания (спит менее 20 секунд)
I	Idle	Процесс бездействует (спит более 20 секунд)
T	sTopping	Остановленный процесс
Z	Zombie	Процесс-зомби. Завершившийся процесс, код возврата которого не считан родителем
B	uninterruptiBle	Непрерывный процесс

Помимо состояния самого процесса можно увидеть и дополнительную информацию (индикатор), которая следует сразу за символом состояния:

Индикатор	Описание
<	Процесс находится в приоритетном режиме
N	Процесс работает в режиме низкого приоритета

L	Процесс Real-time, имеющий страницы, заблокированные в памяти
s	Процесс является лидером сессии
W	Процесс в SWAP
+	Процесс работает на переднем плане

Чтобы посмотреть историю запущенных процессов, можно воспользоваться командой:

```
$ pstree
```

Существуют и другие способы мониторинга процессов — **top** и **htop**. Обязательно ознакомьтесь с ними!

Запуск процессов в ОС Linux

Разберем порядок запуска ОС от нажатия кнопки питания до запуска процессов. В разных ОС чем больше номер шага, тем больше различий.

Порядок запуска при нажатии кнопки питания следующий:

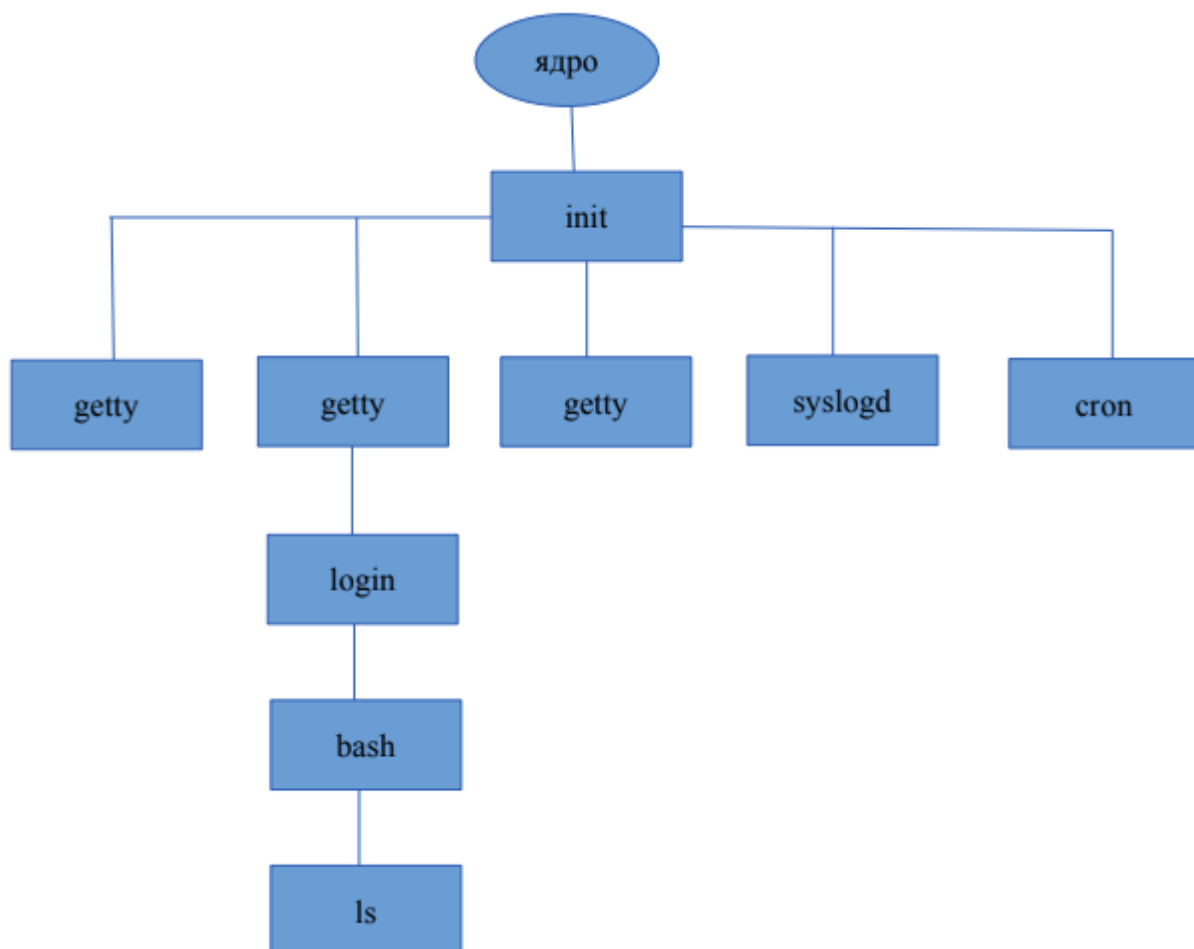
1. **BIOS** — базовая система ввода-вывода. При нажатии кнопки питания подается питание на материнскую плату, либо, при управлении питанием ACPI, на отдельные узлы. Процессор начинает читать код, прошитый в ПЗУ, выполняет программу первичной диагностики POST, инициализирует базовую систему ввода-вывода, после чего читает настройки из энергонезависимой памяти (питаемой от батарейки, установленной на системной плате). Выяснив порядок загрузки, BIOS считывает начальную загрузочную запись диска в память и передает ей управление. В современных компьютерах BIOS заменен на UEFI, которому не требуется читать загрузочную запись диска — так как загрузчик уже входит в UEFI, который считывает заменившую MBR таблицу GPT.
2. **MBR** — начальная загрузочная запись. Содержит данные о физическом носителе и файловой системе, а также программу загрузки. Ее задача — передать управление загрузчику. (В редких случаях загрузчик не требовался. Так в DOS управление передавалось файлу io.sys, размещавшемуся в начале раздела.) В современных системах с UEFI вместо BIOS + MBR используется UEFI+GPT. GPT позволяет адресовать большее пространство, большее количество разделов и уже не содержит загрузчик, который перенесен в UEFI. Тот читает GPT, находит на диске загрузчик операционной системы и передает управление ему.
3. **Загрузчик операционной системы** — его задача обнаружить ядро операционной системы и передать ему управление. ntldr — для Windows, для Linux — lilo (в прошлом), grub2 (в настоящее время).
4. **Ядро операционной системы** — самый важный компонент, работающий в привилегированном режиме процессора и имеющий монопольный доступ к оборудованию и ко всем компонентам ЭВМ. Далее ядро операционной системы запускает два процесса, у которых нет родительских процессов. Для Linux первым процессом пользовательского пространства будет init, а первым процессом ядра — [kthreadd]. Каждый из них является родителем для всех

остальных процессов, то есть `init` — для процессов пользовательского пространства, `[kthreadd]` — процессов ядра.

5. **Процесс `init`** имеет много реализаций. Внешнее поведение `init` одинаковое, но разные реализации имеют особенности и отличия в производительности. Изначально классической реализацией `init` был `Sys V Init`. Некоторое время популярна была система `Upstart` (например, в `Ubuntu 14`). Сейчас в качестве `init` применяется система `System D`, которая и используется по умолчанию в современных дистрибутивах `GNU/Linux`. Независимо от этого, когда мы разбираем взаимодействие процессов и ядра, этот начальный процесс пользовательского пространства именуется как `init`.
6. **В `Linux` процесс `init` является родителем для всех остальных процессов.** `init` стартует сервисы (демоны) `cron` и `syslog`, запускает службы терминалов. Также `init` управляет процессами пользовательского пространства.
7. В зависимости от уровней выстраивания, в особенности это касается процессов-демонов, постоянно выполняющихся в фоновом режиме. полнения запускаются те или иные скрипты, конфигурирующие, запускающие или останавливающие соответствующие демоны.

Пункты 4–6 в других операционных системах (не `Linux`) могут быть другими, но выполняемые действия похожи.

Примерно порядок запуска процессов в `Linux` выглядит так:



Системные вызовы для управления процессами

Чтобы далее разобраться, как происходит создание и совместная работа с процессами, надо узнать, какие есть системные вызовы, предоставляемые ОС Linux (и другими POSIX-совместимыми системами).

fork()

fork() создает новый процесс из существующего. Когда делается системный вызов fork(), программа как бы раздваивается. Одна продолжает с того места, где была, вторая является ее полным клоном и тоже продолжает с того же места. Все переменные, ресурсы остаются прежними. Но обычно развитие в программах идет по-разному. Потому что fork() возвращает значение, и в процессе-родителе оно будет равно id дочернего процесса (PID), а в процессе-дочке нулю. При этом если дочерний процесс имеет возможность получить PPID, то есть id родителя, то у родительского процесса такой возможности нет, разве что сразу после вызова fork сохранить возвращенное значение PID дочернего процесса.

fork() в Linux отличается от create_process() в Windows тем, что fork не может создать процесс из ничего. fork() вызывается существующим процессом и создает полный клон исходного процесса.

exec()

Если fork() используется для распараллеливания выполнения одной задачи, например для увеличения количества входящих соединений, как в простейшей реализации echo-сервера с fork, ssh-сервера или apache-сервера, — то код, выполняемый в дочерних процессах, действительно может быть идентичен.

Если же мы хотим запустить другую программу параллельно с первой, то на первом этапе мы имеем всего лишь две параллельно выполняющихся исходных программы. Тогда нам нужно в дочернем процессе заменить код программы. Для этого используется системный вызов exec()

exec() заменяет исполняемый программный код процессора, в качестве аргумента принимает программу, которую загрузит в процесс.

Теперь у нас появился новый процесс — программа, обеспеченная ресурсами. Она может запросить ресурсы, через malloc() запросить дополнительную память, повысить приоритет процесса.

После того как программа отработала, она сообщает процессу-родителю, с ошибками это было или без. Программа-родитель определяет это на основе кода возврата. Когда процесс освободит все ресурсы, он вернет код возврата в таблицу процессов. В этот момент в ОС в таблице процессов содержится информация о завершившемся процессе, его название и код возврата.

wait()

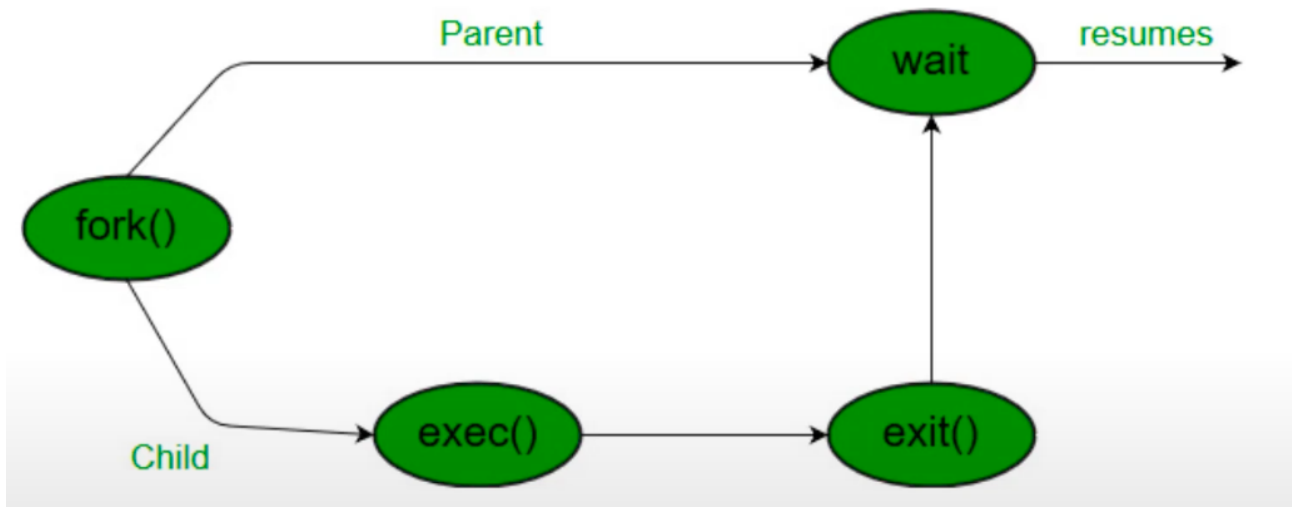
Код возврата считывается системным вызовом wait() процесса-родителя — для получения информации о завершении процесса-потомка. После вызова wait() процесс-потомок полностью удаляется из системы. С этого момента можно считать, что процесс полностью отработал и информацию о том, как он это сделал, мы получили.

Жизненный цикл нового процесса

Процесс создается вызовом fork(). Далее процесс-потомок выполняет exec()).

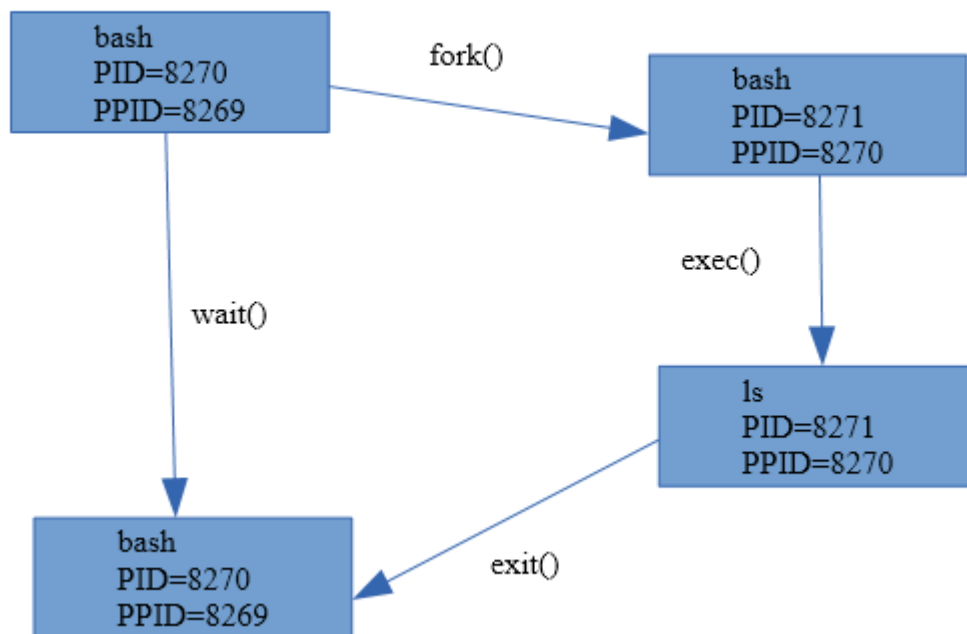
С помощью exec(), в аргументе которого указана программа, в дочернем процессе замещается код родительского процесса на код новой программы. Когда программа отработала, она завершается с помощью exit(), возвращает код возврата.

Родительская программа с помощью `wait()` получает код возврата, а дочерний процесс окончательно перестает существовать.



Пример жизненного цикла

Рассмотрим порядок запуска процесса. Работая в оболочке, мы запустили команду `ls`:



На первом этапе оболочка выполняет системный вызов **`fork()`**, в результате которого происходит ее клонирование: создается полная копия процесса оболочки, включая копию адресного пространства и контекста. Но есть одно отличие между двумя клонами: в процессе-родителе `fork()` возвращает PID потомка, а в потомке код возврата — 0. Каждый из клонов начинает выполнение с момента вызова `fork`, но уже самостоятельно и с учетом вышесказанного.

Потомок, выяснив, что он наследник (так как код возврата 0), с помощью системного вызова **exec** загружает в свое адресное пространство исполняемый код запускаемой программы (в нашем случае **ls**) и далее ее выполняет.

В это же время родительский процесс ждет завершения дочернего процесса с помощью системного вызова **wait()** (если, конечно мы не запустили команду с **&** в конце — в этом случае ожидания не будет).

Дочерний процесс завершается функцией **exit()**. В случае успешного завершения, **exit** возвращает 0, а если произошла ошибка — код ошибки, отличный от нуля. После этого ядро освобождает ресурсы, которые занимал потомок, и передает код его завершения родителю в качестве кода возврата для **wait()**.

Процесс-зомби и процесс-сирота

В отображении программ мониторинга системных ресурсов **ps** и **top** есть такой статус процесса — **Z (Zombie)** — процесс-зомби. Такой статус имеют процессы, которые уже завершились. Статус зомби — обязательная стадия любого процесса в Linux. Процесс-зомби — это процесс, освободивший все ресурсы и ожидающий считывание кода возврата. Мы не можем потерять код возврата, пока его не считал процесс-родитель. Но мы не можем и оставлять занятыми все ресурсы, потому что процесс завершил работу.

Если вы видите процесс-зомби в списке, о нем надо знать следующее. Процесс завершил свое выполнение. Он не обращается к системным вызовам, не использует оперативную память и жесткий диск. Он освободил ресурсы, память, не требует процессорного времени. По факту исполняемой программы уже нет, есть только запись в таблице процессов.

Именно потому, что фактически процесса уже нет, мы никак не можем убить (завершить) такой процесс. Устранить можно только запись в таблице процессов.

Следующий интересный вид процесса — **процесс-сирота**. Бывает, что процесс-родитель прекращает существование, не успев дожидаться завершения **wait()**, то есть раньше завершения потомка. Тогда последний называют процессом-сиротой. Такие процессы усыновляет **init** (в современных версиях **Systemd**).

Процесс-сирота — краткосрочное явление, и увидеть его мы не можем (в отличие от зомби, которые могут существовать долго).

Процессы усыновляются, потому что если у процесса не будет родителя, он не сможет завершить работу. Он освободит ресурсы, вернет код возврата, а так как родителя нет, некому будет считать этот код и процесс будет существовать как зомби.

Наличие процессов-зомби свидетельствует, что:

1. У нас сильно нагруженная система. Между завершением работы процесса и считыванием его кода возврата проходит достаточное время, и мы можем видеть наличие процесса-зомби несколько секунд или минут.
2. Либо есть плохо работающий процесс, который порождает дочерние, но не считывает их коды возврата с помощью **wait()**.

Если процессов-зомби более 1000, ОС будет тратить больше времени на переключение между ними. При этом высоконагруженными считаются системы, в которых работает порядка 1500 процессов. Максимальное количество процессов в Linux — 65535. Такое значение никогда не достигается.

Чтобы устранить процесс-зомби, нужно убить процесс-родитель. Тогда процессы-зомби станут сиротами, будут усыновлены **init**, который с помощью **wait()** получит коды возврата и записи в таблице процессов исчезнут.

Мониторинг ресурсов с помощью top

Консольная утилита **top** служит для динамического просмотра списка процессов. Утилита каждые несколько секунд выдает топ-20 процессов, отсортированных по умолчанию по потреблению процессорного времени. Можно сортировать список процессов по объему занимаемой памяти, утилизации процессора, пользователям или номерам процессов.

```
top - 08:42:22 up 10 days, 21:52, 1 user, load average: 0,00, 0,00, 0,00
Tasks: 139 total, 1 running, 138 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0,0 us, 0,0 sy, 0,0 ni,100,0 id, 0,0 wa, 0,0 hi, 0,0 si, 0,0 st
КиБ Mem : 1016024 total, 288160 free, 179748 used, 548116 buff/cache
КиБ Swap: 1949692 total, 1949396 free, 296 used. 644388 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1223	mysql	20	0	1109684	134020	15376	S	0,3	13,2	4:10.81	mysqld
1	root	20	0	119972	6104	3988	S	0,0	0,6	0:04.83	systemd
2	root	20	0	0	0	0	S	0,0	0,0	0:00.04	kthreadd
3	root	20	0	0	0	0	S	0,0	0,0	0:00.49	ksoftirqd/0
5	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	kworker/0:0H
7	root	20	0	0	0	0	S	0,0	0,0	0:07.42	rcu_sched
8	root	20	0	0	0	0	S	0,0	0,0	0:00.00	rcu_bh
9	root	rt	0	0	0	0	S	0,0	0,0	0:00.00	migration/0
10	root	rt	0	0	0	0	S	0,0	0,0	0:05.16	watchdog/0
11	root	20	0	0	0	0	S	0,0	0,0	0:00.00	kdevtmpfs
12	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	netns
13	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	perf
14	root	20	0	0	0	0	S	0,0	0,0	0:00.21	khungtaskd
15	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	writeback
16	root	25	5	0	0	0	S	0,0	0,0	0:00.00	ksmd
17	root	39	19	0	0	0	S	0,0	0,0	0:00.00	khugepaged
18	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	crypto
19	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	kintegrityd
20	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	bioaset
21	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	kblockd
22	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	ata_sff
23	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	md
24	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	devfreq_wq
28	root	20	0	0	0	0	S	0,0	0,0	0:00.22	kswapd0
29	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	vmstat
30	root	20	0	0	0	0	S	0,0	0,0	0:00.00	fsnotify_mark

Вывод программы top

Наиболее интересные в top — первые три строки вывода.

Разберем, что отображается в выводе программы top (см. картинку).

В первой строке мы видим информацию о текущем времени и **Uptime**. Посмотрите на картинке, как долго работает сервер? 10 дней, 21 час 52 минуты.

На сервере сейчас находится один пользователь.

Load average равен нулю. Это среднее количество процессов в очереди на ожидание ресурсов. LA представлен в трех временных интервалах (поэтому на картинке мы видим три нулевых значения, разделенных запятыми): за минуту, за 5 минут и за 15 минут.

Если процессу требуется процессорное время, которого нет в данный момент, или же информация с устройств ввода-вывода, которая еще не попала память, то процесс помещается в очередь на ожидание ресурсов. Чем больше значение Load Average, тем больше процессов на ожидание ресурсов и нагрузка на сервер.

Принято считать, что в нормальном значении Load Average должно быть меньше или равно общему количеству ядер на процессоре машины.

В третьей строке содержится информация о количестве ядер. В строке **%Cpu(s)** содержится средняя информация. Но если нажать кнопку 1 на цифровой клавиатуре, то увидим всю информацию по ядрам.

После нажатия 1 видим, что присутствует только одна строка %Cpu0.

```
top - 08:44:30 up 10 days, 21:54, 1 user, load average: 0,00, 0,00, 0,00
Tasks: 139 total, 1 running, 138 sleeping, 0 stopped, 0 zombie
%Cpu0 :  0,0 us,  0,0 sy,  0,0 ni,100,0 id,  0,0 wa,  0,0 hi,  0,0 si,  0,0 st
```

Это означает, что в системе одно ядро с номером 0, нумерация ядер идет с нуля.

И видим всю информацию по ядру.

- **Cpu0** — 1 ядро, то есть LA должно быть меньше или равно 1, при условии, что ресурс, который необходим процессам, — это процессор.

Далее во второй строке мы видим информацию по процессам:

- 1 процесс выполняется (running);
- 138 процессов в спящем режиме (sleeping), которые находятся в памяти, но ожидают события;
- 0 остановлено (stopping);
- 0 зомби (zombie).

Самая полезная строка — информация о процентном потреблении процессорных ресурсов.

- **us** — сколько % процессорного времени было потрачено в пространстве пользователя;
- **sy** — сколько % процессорного времени было потрачено в пространстве ядра;
- **ni** — количество процессов, приоритет которых был изменен;
- **id** — сколько процессорного времени простаивает без дела;
- **wa** — сколько % процессорного времени потрачено на ожидание устройств ввода-вывода.х;
- **hi** — аппаратные прерывания (от оборудования);
- **si** — программные прерывания (от программ, вызывающих системные вызовы);
- **st** — сколько процессорного времени было украдено (если система работает в виртуальной машине).

Столбец **us** показывает информацию о том, сколько процессорного времени было потрачено в пространстве пользователя, столбец **sy** — сколько потрачено в пространстве ядра. На этом основании мы можем понять, что грузит систему. Либо драйвера, модули ядра, компоненты ядра операционной системы (sy) — либо сервисы в пространстве пользователя или даже прикладные программы (sy)

Процесс **ni** — **nice** — процессы, приоритет которых был изменен.

У каждого процесса есть приоритет (см. столбец PR перечня процессов). У каждого процесса приоритет 20. Чем меньше эта цифра, тем приоритет выше. По умолчанию процесс создается с приоритетом 20, но при работе он может быть скорректирован.

На иллюстрации видим, что процесс **kworker** при создании получил приоритет 20, но был изменен на 0, а NI (nice) стал 20.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	119972	6104	3988	S	0,0	0,6	0:04.83	systemd
2	root	20	0	0	0	0	S	0,0	0,0	0:00.04	kthreadd
3	root	20	0	0	0	0	S	0,0	0,0	0:00.50	ksoftirqd/0
5	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	kworker/0:0H
7	root	20	0	0	0	0	S	0,0	0,0	0:07.42	rcu_sched
8	root	20	0	0	0	0	S	0,0	0,0	0:00.00	rcu_bh

В столбец **ni** попадают процессы, приоритет которых был изменен. Если мы пытаемся сбалансировать систему и запустить приложение с повышенным приоритетом, можем определить, что пошло не так. Если растет число **ni**, то, скорее всего, приложение, которое мы скорректировали, ведет себя некорректно.

Столбец **i** — **idle** — показывает, сколько процессорного времени простаивает без дела. На картинке вывода **top** видно, что **idle** близко к 100 %, то есть сервер практически не загружен.

Столбец **w** — **wait** — показывает, сколько процессорного времени было потрачено на ожидание информации от устройства ввода-вывода. Если процесс запрашивает информацию с диска, то данные не передаются мгновенно. Сначала операционная система начинает их считывать и передавать в память. Пока система считывает данные с устройства ввода-вывода, процессор ничего не делает, ожидает. Процент процессорного времени, потраченного на ожидание, попадает в столбец **w**.

Столбцы **hi** и **si** отвечают за **Hardware Interruption** — аппаратные прерывания и **Software Interruption** — программные прерывания.

Аппаратные прерывания будут расти, если к серверу подключено оборудование и оно запрашивает прерывания слишком часто.

Программные прерывания растут, когда есть приложение, которое постоянно их генерирует, то есть использует системные вызовы, обращается к операционной системе.

Столбец **st** — **stolen** — показывает, сколько процессорного времени было украдено (если система работает в виртуальной машине). Если не равно 0 — сколько было украдено у гостевой хостовой машины. Информативен при KVM, а в Hyper-V параметр не будет меняться, даже если гипервизор загружен на 100 %.

У **top** есть горячие клавиши:

- **h** — вызов справки;
- **k** — ввести PID для команды **kill**;
- **r** — указать PID для изменения параметра **nice**, влияющий на приоритет процесса (**-20** — максимальный приоритет, **19** — минимальный, **0** — по умолчанию. Отрицательный приоритет может задать суперпользователь). Обычный пользователь может только уменьшать приоритет своих процессов, назначая им **nice** от 0 до 19.
- **1** — указать информацию по ядрам;
- **q** — завершить **top**.

Существует более продвинутая консольная утилита для мониторинга и управления процессами — **htop**. Она не установлена по умолчанию, но доступна из репозитория Ubuntu. Перечислим ее возможности в сравнении с **top**:

- позволяет выполнять горизонтальный и вертикальный скроллинг и видеть все процессы и параметры командной строки;
- htop запускается быстрее top, который собирает данные перед тем, как показать список процессов;
- в htop не надо вводить PID для kill, как в top, — все через скроллинг и функциональные клавиши;
- также нет необходимости вводить PID и значение nice;
- в htop поддерживается мышь;
- top идет в системе по умолчанию и более распространен в мире UNIX.

Отправка сигналов процессам

Нажимаем Alt-F1 (или Ctrl-Alt-F1, если работали в X-сервер) или любой другой терминал.

Запустим процесс, который не завершается сразу, а выводит информацию на экран.

Например, просмотр лога: **tailf** будет отображать все сообщения, которые будут добавлять в файл системного лога:


```
$ tailf /var/log/syslog
```

Мы можем завершить этот процесс, нажав Ctrl-C, или приостановить с помощью Ctrl-Z.

Если приостановили с помощью Ctrl-Z, его можно возобновить командой **fg** (foreground).

```
$ fg
```

Разберем, как послать сигнал процессу из другого терминала.

Нажимаем Alt-F2 (или Ctrl-F1t-F2, если работали в X-сервер), смотрим id процесса.

```
$ ps ax| grep tailf
```

Допустим, полученный PID — это 4242.

Процессу можно направить сигнал: например, 9 — принудительно снять.

```
$ kill -9 4242
```

В Linux есть следующие сигналы:

- **1 (SIGHUP)** — информирует программу о потере связи с терминалом. Эта ситуация часто возникала в прошлом, в режиме терминального доступа. В настоящее время она может применяться для двух целей:
 - информирование дочернего процесса о завершении родительского. При завершении родительского процесса ядро направит сигнал 1 дочерним процессам.

И если приложение запущено в фоновом режиме (background), с помощью указания & в конце команды или **bg** при завершении консоли программа, исполняющаяся в фоновом режиме, также получит сигнал SIGHUP, если она была запущена таким образом. Это очень напоминает исходное значение SIGHUP:

```
$ someprogram&
```

или

```
$ someprogram
^Z
$ bg
```

Этому можно воспрепятствовать, указав **nohup**:

```
$ nohup someprogram&
```

Очевидно, что по умолчанию сигнал приводит к завершению программы. Однако демоны перехватывают его и используют для перечитывания файлов конфигурации.

- **2 (SIGINT)** — формируется при нажатии Ctrl-C, завершает программу, но может перехватываться или блокироваться: например, в программах, в которых Ctrl-C используются для операции копирования или в принципе нет необходимости в таком варианте остановки программы.

- **8 (SIGFPE)** — означает ошибку операции с целочисленной арифметикой (переполнение, деление на ноль). Сохраняется дамп памяти.
- **9 (SIGKILL)** — безусловное завершение программы. Сигнал не может быть перехвачен программой, потому что позволяет ее остановить в любом случае (но не позволит снять процесс-зомби).
- **11 (SIGSEGV)** — формируется при попытке программы обратиться к не принадлежащей ей области памяти. Обычно выводится сообщение «Segmentation fault» и сохраняется дамп памяти. Как правило, такое случается в результате ошибок программиста при работе с указателями.
- **15 (SIGTERM)** — вежливая просьба к программе завершить работу. Она может сохранить данные и т. д.

В Linux есть и другие варианты сигналов.

Для отправки сигнала из операционной системы можно использовать команду kill:

```
$ kill -15 4242
```

Процессы и потоки в программировании

Данный раздел разбирается на языке C, и его можно пропустить. Но наиболее низкоуровневую работу с системными вызовами можно наблюдать именно в таком низкоуровневом языке. Поэтому, если есть желание и интерес, изучите и этот материал.

Работа с процессами на C

Самая простая программа «Hello, world!» будет выполнена как процесс.

hello.c:

```
#include <stdio.h>
int main (void)
{
    printf ("Hello, World!\n");
    getchar ();
    return 0;
}
```

Компилируем и запускаем:

```
$ gcc hello.c -o hello
$ ./hello
```

Мы можем запустить команду **ps ax** и отфильтровать процесс по имени, чтобы узнать его PID.

```
$ ps ax|grep hello
```


Теперь можем послать сигнал процессу, например принудительно завершить его исполнение:

```
$ kill -9 4810
```

(Здесь подразумевается, что 4810 — id процесса **hello**)

Каждый процесс имеет PID — идентификатор процесса. Также у него есть PPID — идентификатор родительского процесса.

Программа, которая выполнялась в одном процессе, при необходимости параллельной обработки может «раздвоиться». При этом изначально запущенный процесс обозначается как процесс-родитель, а порожденный — процесс-потомок.

Как правило, для ветвления используется функция **fork()**. При выполнении fork операционная система копирует процесс (данные тоже будут скопированы: каждый процесс работает со своими данными). После этого и родительский, и дочерний процессы продолжают выполнение с точки fork. Родительский в качестве кода возврата от fork() получит PID дочернего, а дочерний — ноль. Тем не менее дочерний процесс тоже может узнать PPID — идентификатор родительского с помощью getppid(). При этом, если дочерний процесс всегда может узнать PPID, в родительский значение возвращается один раз — в результате fork().

Интересный пример применения процессов — запуск другой программы через **execvp()**.

Если запустить программу в консоли bash (например, ssh) родительская программа словно бы приостанавливается. Происходит это потому, что выполнение ssh происходит в том же процессе, что и bash. Конечно, еще есть терминал, но даже если бы в bash запустили программу, обрабатывающую данные, нам пришлось бы ждать выполнения.

Формат вызова функции **execvp()**:

```
int execvp(const char *file, char *const argv[]);
```

Первый аргумент — имя программы. Поиск осуществляется с учетом переменной окружения PATH.

Второй — перечень аргументов, как и в функции **main()**.

Дочернюю программу необходимо запускать в дочернем процессе. Выполним это на примере запуска программы **ls**:

```
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
int main(int argc, char * argv[])
{ int pid, status;
  if (argc < 2) {
    printf("Usage: %s command, [arg1 [arg2]...]\n", argv[0]);
    return EXIT_FAILURE;
  }
  printf("Starting %s...\n", argv[1]);
  pid = fork();
  if (pid == 0) {
    execvp(argv[1], &argv[1]);
    perror("execvp");
  }
```

```

return EXIT_FAILURE; // Never get there normally
} else {
if (wait(&status) == -1) {
perror("wait");
return EXIT_FAILURE;
}
if (WIFEXITED(status))
printf("Child terminated normally with exit code %i\n",
WEXITSTATUS(status));
if (WIFSIGNALED(status))
printf("Child was terminated by a signal %i\n", WTERMSIG(status));
if (WCOREDUMP(status))
printf("Child dumped core\n");
if (WIFSTOPPED(status))
printf("Child was stopped by a signal %i\n", WSTOPSIG(status));
}
return EXIT_SUCCESS;
}

```

Потоки в C

Для создания потока используется функция **pthread_create**.

Функция потока должна иметь заголовок такого вида:

```
void * func_name(void * arg)
```

Аргумент **arg** — указатель, который передается в последнем параметре функции **pthread_create()**.

Для завершения потока используется функция **pthread_exit()**.

Для синхронизации потоков и получения значений используется **pthread_join()**.

Вызов функции **pthread_join()** приостанавливает выполнение вызвавшего ее потока, пока поток, чей идентификатор передан функции в качестве аргумента, не завершит работу.

```

#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <pthread.h>
void * thread_func(void *arg)
{ int i;
int loc_id = * (int *) arg;
for (i = 0; i < 4; i++) {
printf("Thread %i is running\n", loc_id);
sleep(1);
}
}
int main(int argc, char * argv[])
{ int id1, id2, result;
pthread_t thread1, thread2;
id1 = 1;
result = pthread_create(&thread1, NULL, thread_func, &id1);
if (result != 0) {
perror("Creating the first thread");
return EXIT_FAILURE;
}
id2 = 2;
result = pthread_create(&thread2, NULL, thread_func, &id2);
if (result != 0) {
perror("Creating the second thread");
return EXIT_FAILURE;
}
result = pthread_join(thread1, NULL);
if (result != 0) {
perror("Joining the first thread");
return EXIT_FAILURE;
}
result = pthread_join(thread2, NULL);
if (result != 0) {
perror("Joining the second thread");
return EXIT_FAILURE;
}
printf("Done\n");
return EXIT_SUCCESS;
}

```

Вызывая **pthread_create()** дважды, мы оба раза передаем в качестве третьего параметра адрес функции **thread_func**, в результате чего два созданных потока будут выполнять одну и ту же функцию. Разумеется, не каждая функция может корректно работать таким образом.

Функция, вызываемая из нескольких потоков одновременно, должна обладать свойством реентерабельности. Реентерабельная функция может быть вызвана повторно, когда уже вызвана. Такие функции используют локальные переменные и локально выделенную память, когда их нереентерабельные аналоги могут воспользоваться глобальными переменными. Приведем в пример рекурсивные функции: они также должны обладать свойством реентерабельности.

Компилируем:

```
gcc threads.c -D_REENTRANT -I/usr/include/nptl -L/usr/lib/nptl -lpthread -o threads
```

Практическое задание

Ответьте на следующие вопросы в Google Docs, максимально подробно объясняя, почему вы так считаете. Приложите ссылку на документ к уроку.

1. Чем отличается процесс от программы?
2. Что показывает **%wa** в утилите **top** в ОС Linux?
3. Что такое **Load Average**?
4. Что такое процесс-зомби?
5. Что такое процесс-сирота?
6. Как убить процесс-зомби?
7. Потребляют ли процессы-зомби ресурсы?
8. Что такое процесс **init**?
9. Что делает системный вызов **fork()**?
10. Что делает системный вызов **exec()**?

Задачи со * предназначены для продвинутых учеников.

Дополнительные материалы

1. [Процессы в Linux — йкоманды просмотра и управления.](#)
2. [Ps.](#)
3. [htop и многое другое на пальцах.](#)
4. [Практика работы с сигналами.](#)
5. [Интерактивная система просмотра системных руководств.](#)
6. [Программирование C в Linux — потоки pthreads.](#)

Используемая литература

Для подготовки данного методического пособия были использованы следующие ресурсы:

1. [Процессы в Linux - команды просмотра и управления.](#)
2. [Ps.](#)
3. [htop и многое другое на пальцах.](#)
4. [Радио 86 РК — советский самодельный компьютер..](#)

5. [Многопроцессорность](#).
6. [How to make child process die after parent exits?](#)
7. [Сигналы](#)
8. [Процессы и потоки](#)
9. [Потоки](#).
10. [Summit \(supercomputer\)](#)